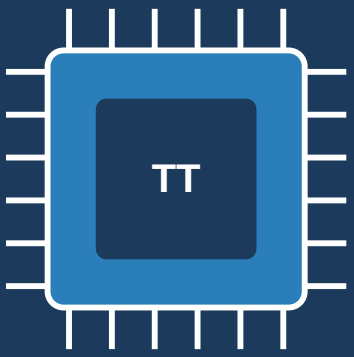
Easy TT CAD

Guide utilisateur



Flow RTL-to-GDS pour Tiny Tapeout

Du code Verilog à la puce, dans une seule interface

Au sommaire

- Installation et premier lancement
- Création et chargement d'un projet
- Les 7 onglets : RTL, testbench, tests, info.yaml, Place & Route, documentation, GitHub
- Tutoriel complet, dépannage et glossaire

Édition d'octobre 2026 - Document rédigé d'après le code actuel du logiciel

Table des matières

1. Présentation . . . 3
1.1 Le parcours en 7 étapes . . . 3
1.2 Points à connaître dès le début . . . 3

2. Prérequis et installation . . . 4
2.1 Système . . . 4
2.2 Outils externes . . . 4
2.3 Installation sur Ubuntu / Debian . . . 4
2.4 Lancer le logiciel . . . 4

3. Découvrir l'interface . . . 5
3.1 Écran d'accueil . . . 5
3.2 La fenêtre principale . . . 5
3.3 Les menus . . . 6
3.4 Console et fenêtre d'attente . . . 6

4. Créer un projet . . . 6
4.1 Ce qui est créé . . . 7

5. Charger un projet existant . . . 7

6. Onglet 1 - RTL (Verilog) . . . 8
6.1 L'assistant de code . . . 8
6.2 Vérifier le Verilog (lint) . . . 9
6.3 Visualiser le schéma RTL . . . 9
6.4 L'éditeur visuel Digital . . . 9

7. Onglet 2 - Testbench (tb.v) . . . 10

8. Onglet 3 - Test (test.py) et simulation . . . 10
8.1 L'assistant de test . . . 10
8.2 Lancer la simulation . . . 11
8.3 Visualiser les chronogrammes . . . 11

9. Onglet 4 - info.yaml . . . 11
9.1 L'assistant de configuration . . . 11

10. Onglet 5 - Place & Route . . . 12
10.1 Lancer le hardening . . . 12
10.2 Ouvrir dans OpenROAD . . . 12

11. Onglet 6 - Documentation . . . 13

12. Onglet 7 - GitHub Tapeout . . . 13
12.1 Avant de commencer . . . 13
12.2 Les champs du formulaire . . . 14
12.3 Envoyer le projet . . . 14
12.4 Étape finale : valider le tapeout . . . 14

13. Tutoriel : de zéro à la puce . . . 14

14. Dépannage . . . 15

Annexe A - Arborescence d'un projet . . . 16

Annexe B - Glossaire . . . 16

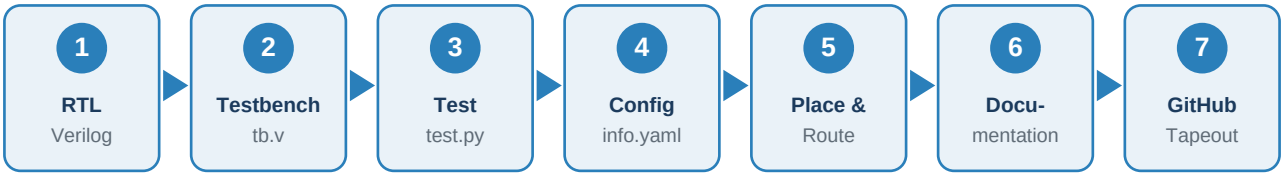
1. Présentation

Easy TT CAD est une application de bureau (Python / Tkinter) qui rassemble, dans une seule fenêtre, toutes les étapes nécessaires pour concevoir une petite puce numérique et la soumettre à la fabrication via le programme **Tiny Tapeout** : écriture du code Verilog, vérification, simulation, placement-routage (génération du fichier GDS), documentation et publication sur GitHub.

Le logiciel masque la ligne de commande (git, venv, pip, Verilator, cocotb, OpenLane/LibreLane…) derrière des boutons et des assistants, tout en affichant chaque commande exécutée dans une console intégrée. Il s'adresse aux débutants, aux étudiants et aux enseignants qui veulent découvrir la conception de circuits intégrés sans maîtriser tous les outils.

1.1 Le parcours en 7 étapes

L'interface est organisée en sept onglets, à parcourir dans l'ordre :



Onglet	Fichier ou ressource	Objectif
1. RTL (Verilog)	`src/project.v`	Décrire le circuit, le vérifier (lint) et visualiser son schéma.
2. Testbench	`test/tb.v`	Relier le circuit au simulateur.
3. Test	`test/test.py`	Écrire les tests (cocotb), lancer la simulation, ouvrir GTKWave.
4. info.yaml	`info.yaml`	Décrire le projet et affecter les broches.
5. Place & Route	dossier du projet	Générer le dessin physique (GDS) : le « hardening ».
6. Documentation	`docs/info.md`	Rédiger la datasheet du projet.
7. GitHub Tapeout	dépôt GitHub	Publier le projet puis le soumettre sur app.tinytapeout.com.

1.2 Points à connaître dès le début

- Le logiciel gère **un seul fichier source Verilog** : `src/project.v`.
- Il est conçu pour **Linux** (il utilise `bash`, `source`, `xdg-open` et `sed`). Sous Windows, utilisez WSL2 avec l'affichage graphique (WSLg).
- Certaines actions **ne sauvegardent pas** automatiquement les éditeurs (simulation, Place & Route, push GitHub). Pensez à utiliser **Fichier > Sauvegarder tout** avant ces étapes.
- Une connexion Internet est nécessaire pour créer un projet, installer Digital et pousser sur GitHub.

2. Prérequis et installation

2.1 Système

Linux (Ubuntu ou Debian recommandés) avec Python 3 et Tkinter. Une résolution d'écran d'au moins 1100 x 850 pixels est conseillée (taille initiale de la fenêtre).

2.2 Outils externes

Easy TT CAD s'appuie sur des programmes installés sur votre machine. Chacun n'est requis que pour la fonction correspondante :

Outil	Rôle	Utilisé par
git	Clonage du template, versionnage, envoi vers GitHub	Création de projet, onglet 7
python3, venv, pip	Création de l'environnement virtuel `tt_env`	Création, simulation, hardening
Icarus Verilog (`iverilog`)	Simulateur Verilog piloté par cocotb	Onglet 3
Verilator	Vérification de la syntaxe (lint)	Onglet 1, bouton Lint
Yosys	Synthèse logique pour le schéma RTL	Onglet 1, Visualiser RTL
netlistsvg (npm)	Conversion de la netlist en image SVG	Onglet 1, Visualiser RTL
GTKWave	Visualisation des chronogrammes	Onglet 3
Java (JRE), unzip, curl	Téléchargement et lancement de l'éditeur Digital	Onglet 1, éditeur visuel
xdg-open	Ouverture du schéma SVG	Onglet 1, Visualiser RTL
Navigateur web	Liens d'aide, page Actions GitHub, app Tiny Tapeout	Menu Aide, onglet 7

2.3 Installation sur Ubuntu / Debian

```
sudo apt update
sudo apt install git python3 python3-venv python3-pip python3-tk \
     iverilog verilator yosys gtkwave default-jre unzip curl \
     xdg-utils nodejs npm
sudo npm install -g netlistsvg
```

> **Information**
> Les paquets Python du flow (cocotb, pytest, lark, librelane et les dépendances de *tt-support-tools*) sont installés **automatiquement** dans l'environnement virtuel `tt_env` au moment de la création d'un projet. Selon votre système, le flow RTL-to-GDS peut nécessiter des composants supplémentaires (PDK, Docker…) : consultez la documentation Tiny Tapeout.

2.4 Lancer le logiciel

```
python3 easy_tt_cad.py
```

(remplacez le nom par celui de votre fichier Python). La fenêtre `Easy TT CAD - Flow RTL-to-GDS` s'ouvre sur un écran d'accueil.

3. Découvrir l'interface

3.1 Écran d'accueil

Au démarrage, un message vous invite à aller dans **Fichier > Nouveau** ou **Fichier > Charger** pour initialiser l'environnement de conception. Les onglets n'apparaissent qu'une fois un projet créé ou chargé.

3.2 La fenêtre principale

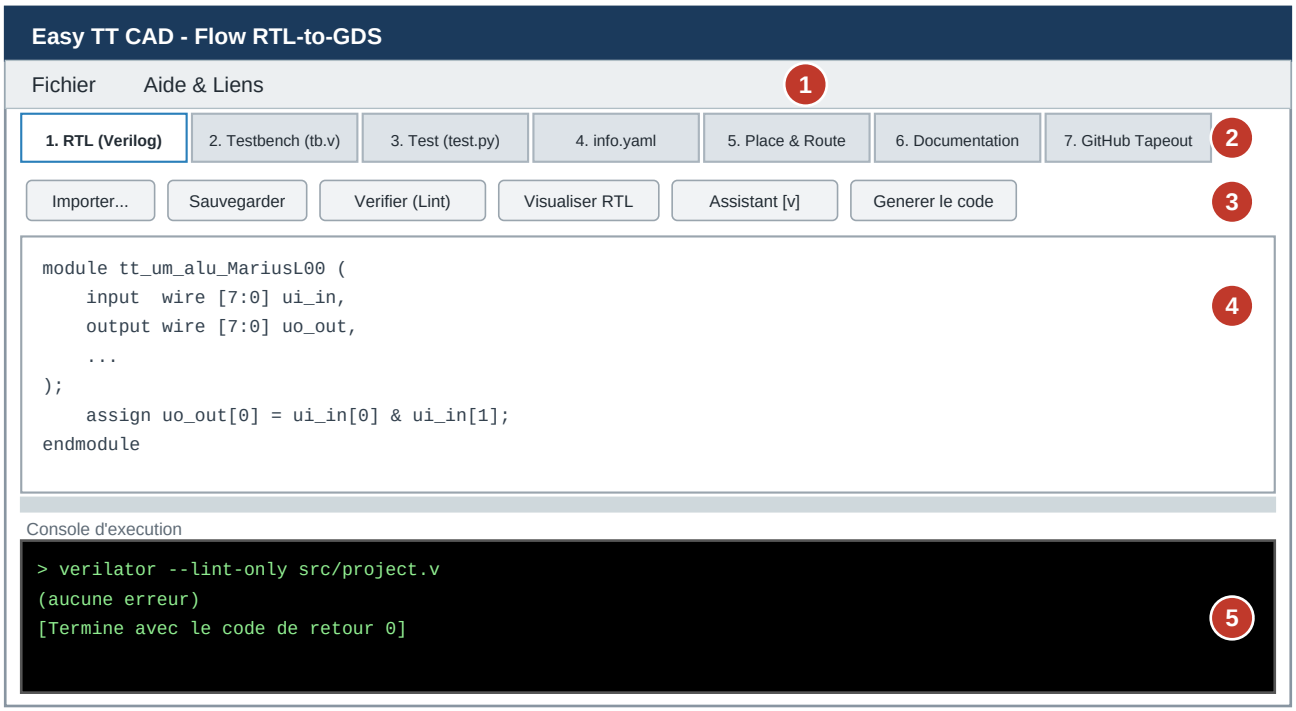


Figure 1 - Disposition de la fenêtre principale (schéma simplifié).

Repère	Zone	Description
1	Barre de menus	Menus **Fichier** et **Aide & Liens**.
2	Onglets	Les 7 étapes du flow, numérotées de 1 à 7.
3	Barre d'outils	Boutons et assistants propres à l'onglet courant.
4	Éditeur	Zone de texte contenant le fichier de l'onglet (police à chasse fixe).
5	Console d'exécution	Journal en temps réel des commandes (fond noir, texte vert).

La séparation entre la zone d'onglets et la console se déplace à la souris : vous pouvez agrandir la console pour suivre un long calcul, ou agrandir l'éditeur pour écrire du code.

3.3 Les menus

Menu	Action
Fichier > Nouveau > Projet	Crée un nouveau projet à partir d'un template Tiny Tapeout.
Fichier > Charger un projet...	Ouvre un dossier de projet existant et remplit les éditeurs.
Fichier > Sauvegarder tout	Enregistre les 5 fichiers éditables : `src/project.v`, `test/tb.v`, `test/test.py`, `info.yaml`, `docs/info.md`.
Fichier > Quitter	Ferme l'application (sans sauvegarde automatique).
Aide & Liens > Documentation Tiny Tapeout	Ouvre tinytapeout.com dans le navigateur.
Aide & Liens > Soumettre son projet (App)	Ouvre app.tinytapeout.com.

3.4 Console et fenêtre d'attente

Chaque commande lancée par le logiciel est affichée dans la console, précédée du signe `>`, puis sa sortie défile en direct. À la fin, la ligne `[Terminé avec le code de retour N]` indique le résultat : **0 signifie succès**, toute autre valeur signale une erreur. Selon l'action, une boîte de dialogue *Succès* ou *Erreur* s'affiche aussi.

Pour les opérations longues (création de projet, premier lancement de Digital, hardening, push GitHub), une fenêtre **« Veuillez patienter »** bloque l'interface jusqu'à la fin du traitement. Elle n'a pas de bouton de fermeture : attendez simplement qu'elle disparaisse.

4. Créer un projet

Chemin : **Fichier > Nouveau > Projet**

1. Une fenêtre de sélection de dossier s'ouvre : choisissez le **dossier de destination**. Le projet sera créé dans un sous-dossier portant le nom du projet.
2. Dans la fenêtre **Configuration du projet**, renseignez les champs (voir tableau ci-dessous).
3. Cliquez sur **[Initialiser le projet]**. Le nom et le pseudo sont obligatoires.
4. Patientez pendant le téléchargement du template et l'installation de l'environnement (quelques minutes selon votre connexion). Un message confirme que le projet est prêt.

Champ	Valeur par défaut	Explication
Nom du projet	`alu`	Nom du dossier et partie centrale du nom du module Verilog.
Pseudo GitHub	`MariusL00`	Votre identifiant GitHub ; utilisé dans le nom du module, pour la configuration git et comme auteur.
Technologie	SkyWater 130nm	Détermine le template Tiny Tapeout cloné (voir ci-dessous).
URL du dépôt	(vide)	Visible uniquement avec « Lien personnalisé » : adresse d'un dépôt template GitHub de votre choix.

Choix de technologie	Template utilisé
SkyWater 130nm (ex : tt08)	`github.com/TinyTapeout/tt08-verilog-template`
GlobalFoundries 180nm (ex : GF26)	`github.com/TinyTapeout/tt-gf180-template`
Lien personnalisé	l'URL que vous saisissez

4.1 Ce qui est créé

Le **module principal** (top module) de votre projet est nommé automatiquement `tt_um_<nom>_<pseudo>`, par exemple `tt_um_alu_MariusL00`. Le logiciel exécute ensuite, dans l'ordre :

- clonage du template, copie de son contenu dans le dossier du projet, puis suppression de son historique git ;
- initialisation d'un dépôt git neuf et ajout du sous-module *tt-support-tools* dans le dossier `tt/` ;
- ajout de `tt_env/` au fichier `.gitignore`, configuration de git (nom, e-mail) et commit initial ;
- création de l'environnement virtuel `tt_env` et installation des paquets Python nécessaires.

Astuce
Utilisez uniquement des **lettres et des chiffres** dans le nom du projet, sans underscore ni tiret, et évitez les tirets dans le pseudo : le nom du module doit rester un identifiant Verilog valide, et l'assistant info.yaml déduit titre et auteur du nom du module en le découpant aux underscores.

Attention
Si un dossier du même nom existe déjà dans le dossier de destination, il est réutilisé et ses fichiers peuvent être écrasés. Choisissez un nom de projet inédit.

Information
À la fin de la création, les éditeurs sont **vides** : c'est normal. Utilisez les assistants de chaque onglet pour générer le code. Ne rechargez pas le projet immédiatement : le fichier `info.yaml` du template contient encore le nom d'exemple, et le logiciel lirait un mauvais nom de module. Générez d'abord votre `info.yaml` avec l'assistant de l'onglet 4.

5. Charger un projet existant

Chemin : **Fichier > Charger un projet...**

1. Sélectionnez le dossier racine du projet (celui qui contient `info.yaml`).
2. Le logiciel lit la ligne `top_module:` de `info.yaml` pour retrouver le nom du module. Si le fichier est absent, le nom `tt_um_inconnu` est utilisé.
3. Les cinq fichiers sont chargés dans les éditeurs : `src/project.v`, `test/tb.v`, `test/test.py`, `info.yaml` et `docs/info.md`. Un fichier manquant laisse simplement l'éditeur correspondant vide.
4. Un message de confirmation et une ligne dans la console indiquent que le projet est chargé.

Attention
La simulation, le Place & Route et le push utilisent l'environnement `tt_env` et le dossier `tt/` à la racine du projet. Un projet créé avec Easy TT CAD les contient déjà ; pour un projet créé autrement, vous devez les préparer vous-même.

6. Onglet 1 - RTL (Verilog)

Cet onglet contient le code de votre circuit, enregistré dans `src/project.v`.

Bouton	Fonction
[Importer un fichier...]	Remplace le contenu de l'éditeur par un fichier de votre choix (par exemple exporté depuis Digital).
[Sauvegarder src/project.v]	Enregistre l'éditeur dans le fichier du projet.
[Vérifier le Verilog (Lint)]	Contrôle le nom du module puis lance Verilator.
[Visualiser RTL (Schéma)]	Génère et ouvre un schéma du circuit.
Assistant de code + **[Générer le code]**	Insère un modèle Verilog prêt à l'emploi.
[Ouvrir l'éditeur visuel (Digital)]	Lance l'outil de dessin de circuits Digital.

6.1 L'assistant de code

Choisissez un modèle dans la liste, puis cliquez sur **[Générer le code]**. Le logiciel demande confirmation car **le contenu actuel de l'éditeur est remplacé**. Les quatre modèles partagent la même déclaration de module, avec les ports standard Tiny Tapeout :

Port	Sens	Largeur	Rôle
`ui_in`	entrée	8 bits	Entrées dédiées (broches ui).
`uo_out`	sortie	8 bits	Sorties dédiées (broches uo).
`uio_in`	entrée	8 bits	Broches bidirectionnelles, lues en entrée.
`uio_out`	sortie	8 bits	Broches bidirectionnelles, écrites en sortie.
`uio_oe`	sortie	8 bits	Direction de chaque broche uio : 1 = sortie, 0 = entrée.
`ena`	entrée	1 bit	Signal d'activation (actif à 1).
`clk`	entrée	1 bit	Horloge.
`rst_n`	entrée	1 bit	Remise à zéro, active à 0.

Dans tous les modèles, les broches bidirectionnelles sont configurées en entrées (`uio_oe = 8'b00000000`). Les modèles disponibles :

Modèle	Comportement
1. Squelette de base (vide)	Déclaration du module et sorties forcées à 0 ; à compléter avec votre logique.
2. Hello World (portes ET/OU)	`uo_out[0] = ui_in[0] & ui_in[1]` et `uo_out[1] = ui_in[2] | ui_in[3]`. Circuit purement combinatoire.
3. Compteur 8 bits	Compteur synchrone, reset asynchrone actif à 0, incrémenté à chaque front d'horloge quand `ena` vaut 1 ; valeur envoyée sur `uo_out`.
4. Clignotant (Blink LED)	Diviseur d'horloge 24 bits ; le bit de poids fort (`uo_out[0]`) change lentement pour faire clignoter une LED.

Exemple : le modèle *Hello World* pour le projet `tt_um_alu_MariusL00` (en-tête abrégé) :

```
module tt_um_alu_MariusL00 (
    input  wire [7:0] ui_in,    // Entrées dédiées
    output wire [7:0] uo_out,   // Sorties dédiées
    ... (uio_in, uio_out, uio_oe, ena, clk, rst_n)
);
    assign uio_oe  = 8'b00000000;
    assign uio_out = 8'b00000000;

    assign uo_out[0] = ui_in[0] & ui_in[1];
    assign uo_out[1] = ui_in[2] | ui_in[3];
    assign uo_out[7:2] = 6'b000000;
endmodule
```

6.2 Vérifier le Verilog (lint)

1. Le logiciel vérifie que le code contient la ligne `module <top_module>`. Sinon, un avertissement affiche la ligne attendue et l'opération s'arrête.
2. Le fichier `src/project.v` est sauvegardé automatiquement.
3. La commande `verilator --lint-only src/project.v` est exécutée.
4. Succès : un message confirme l'absence d'erreur de syntaxe. Échec : consultez la console, qui indique fichier, ligne et nature de l'erreur.

6.3 Visualiser le schéma RTL

Ce bouton sauvegarde le fichier, puis enchaîne **Yosys** (synthèse logique vers `rtl_netlist.json`) et **netlistsvg** (conversion en `rtl_view.svg`). Un fond blanc est ajouté à l'image, qui s'ouvre ensuite dans votre visionneuse par défaut. Les deux fichiers sont placés à la racine du projet.

6.4 L'éditeur visuel Digital

Digital (H. Neemann) est un simulateur de circuits logiques graphique. Le bouton le lance depuis `~/.easy_tt_cad/Digital/Digital.jar`.

- **Premier lancement** : Digital est téléchargé depuis GitHub puis décompressé (une seule fois). Java, unzip et curl doivent être installés.
- **Lancements suivants** : l'outil démarre directement.
- **Flux de travail** : dessinez votre circuit dans Digital, exportez-le en Verilog avec les fonctions d'export de Digital, puis utilisez **[Importer un fichier...]** dans l'onglet 1.

> **Astuce**
> Le module exporté par Digital doit porter le nom exact du top module (`tt_um_<nom>_<pseudo>`) et exposer les ports Tiny Tapeout. Adaptez-le dans l'éditeur avant de lancer le lint.

7. Onglet 2 - Testbench (tb.v)

Le **testbench** est un petit module Verilog qui instancie votre circuit et expose ses signaux au simulateur Python (cocotb). Il est enregistré dans `test/tb.v`.

Bouton	Fonction
[Générer tb.v automatiquement]	Écrit un testbench complet adapté à votre top module, puis le sauvegarde.
[Importer un fichier...]	Charge un testbench existant dans l'éditeur.
[Sauvegarder test/tb.v]	Enregistre l'éditeur.

Le fichier généré configure l'enregistrement des signaux dans `tb.vcd` (pour GTKWave), déclare les signaux `clk`, `rst_n`, `ena`, `ui_in`, `uio_in`, `uo_out`, `uio_out`, `uio_oe` et instancie votre module sous le nom `user_project`. Extrait :

```
module tb ();
    initial begin
        $dumpfile("tb.vcd");
        $dumpvars(0, tb);
        #1;
    end
    reg clk;  reg rst_n;  reg ena;  reg [7:0] ui_in;  reg [7:0] uio_in;
    wire [7:0] uo_out;  wire [7:0] uio_out;  wire [7:0] uio_oe;

    tt_um_alu_MariusL00 user_project (
        .ui_in(ui_in), .uo_out(uo_out), .uio_in(uio_in),
        .uio_out(uio_out), .uio_oe(uio_oe),
        .ena(ena), .clk(clk), .rst_n(rst_n)
    );
endmodule
```

> **Information**
> Vous n'avez normalement **pas besoin de modifier** ce fichier : l'horloge et les stimuli sont pilotés depuis `test.py`. Régénérez-le si vous changez le nom du top module.

8. Onglet 3 - Test (test.py) et simulation

Les tests sont écrits en Python avec **cocotb** et enregistrés dans `test/test.py`. Le simulateur Icarus Verilog exécute votre circuit pendant que le script applique des entrées et lit les sorties.

8.1 L'assistant de test

Choisissez un modèle puis cliquez sur **[Générer]** (confirmation demandée, le contenu actuel est remplacé, puis le fichier est sauvegardé automatiquement).

Modèle	Contenu
1. Test Logique (ex : Portes, ALU)	Initialise les entrées, applique la valeur 3 sur `ui_in`, attend 1 ns puis affiche la sortie. Adapté aux circuits combinatoires.
2. Test Séquentiel (ex : Compteur, Horloge)	Démarre une horloge (période 10 µs), applique un reset de 5 cycles, le relâche, laisse tourner 10 cycles puis affiche la sortie.

> **Attention**
> Dans le modèle *Test Logique*, la ligne `assert` est **en commentaire** : tant que vous ne la décommentez pas et ne l'adaptez pas, le test réussit toujours. De même, le test séquentiel ne fait qu'afficher la valeur finale : ajoutez vos propres `assert` pour vérifier le comportement.

8.2 Lancer la simulation

1. Sauvegardez vos fichiers (**Fichier > Sauvegarder tout**). La simulation ne le fait pas pour vous.
2. Cliquez sur **[Lancer la Simulation (Test)]**.
3. Le logiciel réécrit le fichier `test/Makefile` (Icarus, Verilog, `src/project.v` et `test/tb.v`), puis exécute `make -B` dans l'environnement `tt_env`.
4. Consultez la console : cocotb affiche un résumé avec **PASS** ou **FAIL** pour chaque test.

> **Attention**
> Le `test/Makefile` est **réécrit à chaque simulation** : toute modification manuelle sera perdue. Ce Makefile ne compile que `project.v` et référence la bibliothèque SkyWater pour les simulations de niveau portes.

8.3 Visualiser les chronogrammes

Après une simulation réussie, **[Ouvrir GTKWave]** ouvre `test/tb.vcd`. Dans GTKWave, développez *tb* puis *user_project* dans l'arborescence, sélectionnez des signaux et ajoutez-les (**[Append]** ou glisser-déposer) dans le panneau des formes d'onde.

9. Onglet 4 - info.yaml

Le fichier `info.yaml` décrit votre projet pour Tiny Tapeout : titre, auteur, module principal, fichiers sources et nom de chaque broche.

Bouton	Fonction
[Assistant Configuration & Pinout]	Ouvre un formulaire qui génère le fichier complet.
[Importer un fichier...]	Charge un info.yaml existant.
[Sauvegarder info.yaml]	Enregistre l'éditeur.

9.1 L'assistant de configuration

1. Renseignez le **Titre** (prérempli avec la partie centrale du nom du module) et la **Description**.
2. Dans la grille 3 colonnes x 8 lignes, donnez un nom à chaque broche : **Entrées** `ui[0..7]`, **Sorties** `uo[0..7]`, **Bidir** `uio[0..7]`. Valeurs par défaut : `data_in[i]`, `data_out[i]` et `unused`. Un champ vide devient `unused`.
3. Cliquez sur **[Générer et Sauvegarder info.yaml]** : le fichier est affiché dans l'éditeur et enregistré.

Extrait du fichier produit :

```
project:
  title:        "alu"
  author:       "MariusL00"
  description:  "Description de mon circuit"
  language:     "Verilog"
  clock_hz:     0
  tiles: "1x1"
  top_module:  "tt_um_alu_MariusL00"
  source_files:
    - "project.v"
pinout:
  ui[0]: "data_in[0]"
  ...
  uo[0]: "data_out[0]"
  ...
  uio[0]: "unused"
yaml_version: 6
```

> **Attention**
> L'assistant **réécrit tout le fichier**. Les valeurs spéciales ajoutées à la main sont perdues (taille du projet `tiles`, fréquence `clock_hz`, identifiant Discord, liste de fichiers sources). Retouchez-les ensuite directement dans l'éditeur, puis sauvegardez.

> **Astuce**
> Si votre circuit utilise une horloge, remplacez `clock_hz: 0` par sa fréquence réelle en hertz (par exemple `10000000` pour 10 MHz).

10. Onglet 5 - Place & Route

Cette étape, appelée **hardening**, transforme votre Verilog en dessin physique de la puce : synthèse, placement des cellules, routage, puis vérifications **DRC** et **LVS**. Le résultat est un fichier **GDS**, le format envoyé à la fonderie.

10.1 Lancer le hardening

1. Vérifiez que `src/project.v` et `info.yaml` sont **sauvegardés** et cohérents (même nom de module).
2. Choisissez la **Technologie (PDK)** dans la liste, en accord avec le template utilisé à la création du projet.
3. Cliquez sur **[Placer et Router (Hardening)]** puis patientez : l'opération dure en général 3 à 5 minutes.
4. À la fin, un message de succès vous invite à contrôler dans la console les mentions **LVS** et **DRC**. Les résultats sont produits dans le dossier du projet (généralement sous `runs/`).

PDK choisi	Variable PDK	Commandes exécutées (dans tt_env)
SkyWater 130nm	`sky130A`	`tt_tool.py --create-user-config` puis `--harden`
GlobalFoundries 180nm	`gf180mcuD`	`tt_tool.py --gf-create-user-config` puis `--gf-harden`
IHP 130nm	`ihp-sg13g2`	`tt_tool.py --ihp-create-user-config` puis `--ihp-harden`

10.2 Ouvrir dans OpenROAD

Le bouton **[Ouvrir dans OpenROAD]** exécute `./tt/tt_tool.py --open-in-openroad` pour inspecter graphiquement le placement et le routage du dernier hardening.

> **Attention**
> Le menu de technologie de cet onglet est **indépendant** du choix fait à la création du projet. Si les deux ne correspondent pas, le hardening risque d'échouer.

11. Onglet 6 - Documentation

La documentation est rédigée en Markdown dans `docs/info.md`. Elle sert à produire la **datasheet** de votre projet, générée par GitHub Actions après le push.

Bouton	Fonction
[Générer Template info.md]	Insère le modèle officiel avec ses trois sections (remplace le contenu actuel).
[Importer un fichier...]	Charge un fichier Markdown existant.
[Sauvegarder docs/info.md]	Enregistre l'éditeur.

Le modèle contient les sections suivantes, que vous devez compléter (supprimez celles qui ne servent pas) :

- **How it works** : expliquez le fonctionnement de votre circuit ;
- **How to test** : expliquez comment l'utiliser et le tester ;
- **External hardware** : listez le matériel externe (PMOD, afficheur, LED…), le cas échéant.

> **Astuce**
> Vous pouvez placer des images dans le dossier `docs/` et les référencer dans le Markdown. Chaque image doit peser **moins de 512 ko**.

12. Onglet 7 - GitHub Tapeout

Cet onglet publie votre projet sur GitHub, où des *GitHub Actions* construisent le GDS et la documentation, puis vous mène vers l'application Tiny Tapeout pour réserver votre place.

12.1 Avant de commencer

1. Créez un **dépôt GitHub** pour votre projet et notez son URL (par exemple `https://github.com/pseudo/mon-projet.git`).
2. Créez un **jeton d'accès personnel** (token) classique : GitHub, **Settings > Developer settings > Personal access tokens > Tokens (classic)**. Cochez au minimum les droits `repo` et `workflow` (indispensable pour envoyer les fichiers de workflow).
3. Dans le dépôt, ouvrez **Settings > Pages** et réglez **Build and deployment > Source** sur **GitHub Actions**.
4. Ouvrez **Settings > Actions > General** et, dans **Workflow permissions**, cochez **Read and write permissions**.

> **Réglages GitHub indispensables**
> Ces deux réglages (Pages et permissions des workflows) sont rappelés en rouge dans l'onglet. Sans eux, les workflows de documentation et de GDS échoueront.

12.2 Les champs du formulaire

Champ	Contenu
Pseudo	Votre identifiant GitHub (prérempli avec la valeur par défaut `MariusL00`, à modifier).
Token (Clé classique)	Le jeton créé à l'étape précédente (saisie masquée par des étoiles).
URL du Repo	Adresse HTTPS du dépôt, commençant par `https://`.
Message du commit	Texte du commit (défaut : « Mon design prêt pour le tapeout »).

12.3 Envoyer le projet

1. Sauvegardez vos fichiers (**Fichier > Sauvegarder tout**) : le push n'enregistre pas les éditeurs.
2. Cliquez sur **[Push to GitHub]**.
3. Le logiciel écrit le workflow `.github/workflows/docs.yaml` (installation de Typst puis construction de la documentation), configure git, ajoute tous les fichiers, crée un commit, renomme la branche en `main`, déclare le dépôt distant et envoie le tout.
4. En cas de succès, votre navigateur s'ouvre sur la page **Actions** du dépôt.
5. Vérifiez que les workflows (documentation, GDS…) se terminent tous par une **icône verte**.

> **Deux points de sécurité**
> Le push utilise l'option **force** : il **écrase** le contenu actuel de la branche `main` du dépôt distant. Utilisez un dépôt dédié à ce projet. De plus, le jeton est intégré à l'adresse du dépôt distant et donc stocké dans `.git/config` : ne partagez pas le dossier `.git` et révoquez le jeton sur GitHub quand vous n'en avez plus besoin. Dans la console, le jeton est masqué (« ******** »).

12.4 Étape finale : valider le tapeout

Une fois les Actions au vert, cliquez sur **[Ouvrir app.tinytapeout.com]**. Sur ce site, connectez-vous avec GitHub, sélectionnez votre dépôt et suivez les étapes de réservation et de paiement pour soumettre officiellement la puce. Les modalités exactes (dates de la navette, tarifs) évoluent : référez-vous au site officiel.

13. Tutoriel : de zéro à la puce

Ce parcours complet crée un **compteur 8 bits** et suit toutes les étapes du logiciel.

1. **Fichier > Nouveau > Projet** : dossier de destination, nom `compteur`, votre pseudo GitHub, technologie SkyWater 130nm, puis **[Initialiser le projet]**. Attendez la fin de l'installation.
2. **Onglet 1** : choisissez *3. Compteur 8 bits*, cliquez sur **[Générer le code]** et confirmez.
3. Cliquez sur **[Vérifier le Verilog (Lint)]** : le message de succès doit apparaître. Essayez aussi **[Visualiser RTL (Schéma)]**.
4. **Onglet 2** : cliquez sur **[Générer tb.v automatiquement]**.
5. **Onglet 3** : choisissez *2. Test Séquentiel* et **[Générer]**. Ajoutez à la fin du test une vérification, par exemple `assert int(dut.uo_out.value) > 0`.
6. **Fichier > Sauvegarder tout**, puis **[Lancer la Simulation (Test)]** : cherchez **PASS** dans la console. Ouvrez **[GTKWave]** pour voir le compteur évoluer.
7. **Onglet 4** : ouvrez l'assistant, nommez les sorties `count[0]` à `count[7]`, laissez le reste en `unused`, générez. Dans l'éditeur, mettez la bonne valeur de `clock_hz`.
8. **Onglet 6** : **[Générer Template info.md]**, complétez les sections, sauvegardez.

9. **Onglet 5** : gardez SkyWater 130nm, cliquez sur **[Placer et Router (Hardening)]**, attendez 3 à 5 minutes et vérifiez DRC/LVS dans la console.
10. **Onglet 7** : réglez Pages et permissions sur GitHub, saisissez pseudo, token et URL, sauvegardez tout, puis **[Push to GitHub]**.
11. Contrôlez que les Actions sont vertes, puis rendez-vous sur app.tinytapeout.com pour valider le tapeout.

14. Dépannage

Symptôme	Cause probable et solution
« Veuillez d'abord créer ou charger un projet »	Aucun projet n'est actif : utilisez le menu **Fichier**.
Avertissement « Erreur de Top Module »	Le code ne contient pas `module <top_module>`. Corrigez le nom du module pour qu'il corresponde à celui du projet, ou régénérez le code avec l'assistant.
Lint ou schéma échoue (code de retour 127)	Programme introuvable : installez Verilator, Yosys, netlistsvg et xdg-utils (voir section 2.3).
Schéma RTL non généré	Vérifiez d'abord que le lint passe ; vérifiez ensuite l'installation de netlistsvg `(npm)`.
Simulation : `tt_env/bin/activate` introuvable	L'environnement virtuel n'existe pas (projet créé hors du logiciel ou installation interrompue). Recréez le projet ou créez `tt_env` à la main.
Simulation : résultat inchangé après modification	Vous n'avez pas sauvegardé : la simulation lit les fichiers sur disque. Utilisez **Sauvegarder tout**.
GTKWave ne trouve pas le fichier	`test/tb.vcd` n'existe qu'après une simulation réussie dont le `tb.v` contient `$dumpfile`.
Digital ne démarre pas	Vérifiez Java, unzip et curl. Pour forcer un nouveau téléchargement, supprimez le dossier `~/.easy_tt_cad/Digital`.
La fenêtre « Veuillez patienter » reste au premier lancement de Digital	Digital tourne en arrière-plan et garde la commande « ouverte » : fermez Digital pour que la fenêtre disparaisse.
Hardening échoue	Lisez la console. Vérifiez `info.yaml` (`top_module`, `source_files`), la sauvegarde de `project.v` et la cohérence entre PDK et template.
Push refusé (authentification)	Vérifiez pseudo, token (droits `repo` et `workflow`), validité du token et URL du dépôt en HTTPS.
Actions GitHub en rouge	Ouvrez le workflow défaillant sur GitHub pour lire le journal. Vérifiez les réglages Pages et Workflow permissions.
Fonctions Linux indisponibles sous Windows/macOS	Le logiciel n'est pas prévu pour ces systèmes : utilisez Linux ou WSL2.

Annexe A - Arborescence d'un projet

```
mon_projet/
|-- info.yaml              description et pinout du projet         (onglet 4)
|-- src/
|   `-- project.v          code Verilog                            (onglet 1)
|-- test/
|   |-- tb.v               testbench                               (onglet 2)
|   |-- test.py            tests cocotb                            (onglet 3)
|   `-- Makefile           réécrit à chaque simulation
|-- docs/
|   `-- info.md            datasheet                               (onglet 6)
|-- .github/workflows/
|   `-- docs.yaml          écrit au moment du push                 (onglet 7)
|-- tt/                    tt-support-tools (outils Tiny Tapeout)
|-- tt_env/                environnement virtuel Python (ignoré par git)
|-- rtl_netlist.json       netlist Yosys (Visualiser RTL)
`-- rtl_view.svg           schéma RTL
```

Dossier global du logiciel : `~/.easy_tt_cad/Digital/` (copie de l'éditeur Digital).

Annexe B - Glossaire

Terme	Définition
RTL	Register Transfer Level : description du circuit en code (ici Verilog).
Verilog	Langage de description matérielle utilisé pour décrire le circuit.
Lint	Vérification statique du code (syntaxe, avertissements) sans simulation.
Testbench	Module de simulation qui entoure le circuit et lui applique des signaux.
cocotb	Bibliothèque Python pour écrire des tests de circuits Verilog.
Chronogramme	Courbes de l'évolution des signaux dans le temps (fichier .vcd, vu dans GTKWave).
Top module	Module de plus haut niveau ; nommé `tt_um_<nom>_<pseudo>` dans Tiny Tapeout.
Tile	Unité de surface de la puce allouée à un projet (champ `tiles`).
PDK	Process Design Kit : données de la technologie de fabrication (SkyWater 130nm, GF 180nm, IHP 130nm).
Hardening	Conversion du RTL en layout physique : synthèse, placement, routage, vérifications.
GDS	Fichier de dessin des masques de la puce, envoyé à la fonderie.
DRC	Design Rule Check : contrôle du respect des règles de dessin de la technologie.
LVS	Layout Versus Schematic : contrôle que le layout correspond bien au circuit décrit.
Tapeout	Envoi du design final à la fabrication.
GitHub Actions	Automatisations exécutées par GitHub (construction du GDS et de la documentation).